

Master Degree in
Applied and Computational Mathematics
2025-2026

Master Thesis

Computational algorithms for pricing financial derivatives under local volatility models

Jose Enrique Zafra Mena

Jorge Morón Vidal

Francisco Manuel Bernal Martínez

Madrid, 2026

AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

Keywords:

DEDICATION

CONTENTS

1. INTRODUCTION.	1
1.1. Motivation	1
1.2. Main Objectives	1
1.3. Key contributions	1
2. MATHEMATICAL AND FINANCIAL BACKGROUND	2
2.1. Mathematical Framework For Option Pricing	2
2.2. Stochastic Calculus Background	2
2.3. The Heston Stochastic Volatility Model	3
2.4. Pricing Via Characteristic Functions	3
2.5. The Fourier-COS method	3
2.5.1. The COS Method Applied To The Heston Model.	4
2.6. Implied Volatility And Numerical Inversion.	4
2.7. State Of The Art	4
3. NEURAL NETWORK PRICER FOR HESTON VANILLA OPTIONS.	5
3.1. Problem Formulation	5
3.2. Synthetic Data Formulation	6
3.2.1. Parameter Ranges And Constraints.	6
3.2.2. Sampling Strategy: Latin Hypercube Sampling	7
3.2.3. Dataset Splitting And Preprocessing	8
3.2.4. Numerical Label Generation (COS + Brent)	8
3.3. Neural Network Architecture	9
3.3.1. Model Class	9
3.3.2. Architecture Details	9
3.3.3. Output Interpretation.	10
3.4. Training Procedure.	10
3.4.1. Loss Function.	11
3.4.2. Optimizer And Hyperparameters	11
3.4.3. Regularization And Callbacks	12

4. CALIBRATION NEURAL NETWORK FOR THE HESTON MODEL.	14
4.1. Calibration problem formulation	14
4.2. Input representation: implied volatility surfaces.	14
4.3. Synthetic calibration dataset.	14
4.4. CaNN architecture	14
5. PINN FOR GREEKS COMPUTATION.	15
6. NEURAL NETWORK PERFORMANCE AND BENCHMARKING.	16
6.1. Pricing Accuracy	16
6.2. Implied Volatility Accuracy	16
6.3. Computational Efficiency	16
6.4. Generalization And Robustness.	16
7. CONCLUSION AND FUTURE WORK	17
7.1. Market Data.	17
7.2. Exotic Derivatives	17
7.3. Greeks And Hedging.	17
7.4. Model Extension	17
7.5. Machine Learning Improvements.	17
BIBLIOGRAPHY.	

LIST OF FIGURES

LIST OF TABLES

3.1	Input ranges used for synthetic data generation	7
3.2	Neural network architecture used for the ANN pricer	10
3.3	Reference and runner-up runs used in optimizer selection	12
3.4	Training hyperparameters of the selected run (Liu_like_v01)	12

1. INTRODUCTION

1.1. Motivation

1.2. Main Objectives

1.3. Key contributions

2. MATHEMATICAL AND FINANCIAL BACKGROUND

2.1. Mathematical Framework For Option Pricing

2.2. Stochastic Calculus Background

TODO: add Cumulative Distribution Function definition

TODO: add Probability Density Function definition (and delete from below)

[Oorsterlee book] The *Characteristic Function* (ChF) for $u \in \mathbb{R}$ of the random variable X is the Fourier-Stieltjes transform of the cumulative distribution function $F_X(x)$.

$$\phi_X(u) := \mathbb{E}[e^{iuX}] = \int_{-\infty}^{\infty} e^{iux} dF_X(x) = \int_{-\infty}^{\infty} e^{iux} f_X(x) dx, \quad (2.1)$$

where $f_X(u)$ is the *Probability Density Function*, given by

$$f_X(u) = \frac{\partial F_X(u)}{\partial u}. \quad (2.2)$$

Applying the logarithm to the ChF, we define the *Cumulant Characteristic Function* by

$$\zeta_X(u) = \log \mathbb{E}[e^{iuX}] = \log \phi_X(u). \quad (2.3)$$

If we compute the MacLaurin expansion to ζ_X , we arrive to

$$\zeta_X(u) = \sum_{k=0}^{\infty} \frac{\partial^k \zeta_X(u)}{\partial u^k} \Big|_{u=0} \frac{u^k}{k!} = \sum_{k=0}^{\infty} \zeta_k \frac{u^k}{k!}, \quad (2.4)$$

where $\zeta_k \equiv \frac{\partial^k \zeta_X(u)}{\partial u^k} \Big|_{u=0}$ is the k -th *cumulant*.

Furthermore, we can define the *Moment Generating Function*, $\mathcal{M}_X(u)$, by

$$\mathcal{M}_X(u) := \phi_X(-iu) = \mathbb{E}[e^{uX}]. \quad (2.5)$$

Since the MGF is $\mathcal{M}_X(u) = \phi_X(-iu)$ and $\zeta_X(u) = \log \mathcal{M}_X(u)$, we can compute the cumulants, using the ChF, as

$$\zeta_k = \frac{\partial^k \zeta_X(u)}{\partial u^k} \Big|_{u=0} = \frac{\partial^k}{\partial u^k} \log \phi_X(-iu) \Big|_{u=0}. \quad (2.6)$$

2.3. The Heston Stochastic Volatility Model

2.4. Pricing Via Characteristic Functions

2.5. The Fourier-COS method

TODO: rellenar

The COS method is the ...

The main reasons we want to use a cosine expansion are:

- In a cosine Fourier expansion we can represent even functions around 0 exactly.
- We can extend any function $g : [0, \pi] \rightarrow \mathbb{R}$ to become an even function on $[-\pi, \pi]$ as follows:

$$\bar{g}(\theta) = \begin{cases} g(\theta), & \theta \geq 0, \\ g(-\theta), & \theta < 0. \end{cases} \quad (2.7)$$

- For functions supported on any other finite interval, say $[a, b] \in \mathbb{R}$, the Fourier cos series expansion can be obtained via a change of variables:

$$\theta := \frac{y-a}{b-a}\pi, \quad y := \frac{b-a}{\pi}\theta + a.$$

TODO: falta más teoría para enlazar

We finally arrive to the approximation of the PDF

$$\hat{f}_X(y) = \sum'_{k=0}^{N-1} \bar{F}_k \cos\left(\pi k \frac{y-a}{b-a}\right), \quad (2.8)$$

where $\sum'$ indicates that the first term should be multiplied by 1/2, and the coefficients depend on the ChF:

$$\bar{F}_k := \frac{2}{b-a} \operatorname{Re} \left[\phi_X \left(\frac{\pi k}{b-a} \right) \cdot \exp \left(-\frac{i\pi a k}{b-a} \right) \right]. \quad (2.9)$$

TODO: no se si poner algo aquí o nueva subsection

The value of a plain vanilla option is given by

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y) f_X(T, y; t_0, x) dy, \quad (2.10)$$

where $\tau = T - t_0$, $f_X(T, y; t_0, x)$ is the transition probability density of $X(t)$, and r the interest rate.

Applying the COS method to the last expression, we arrive to

$$V(t_0, x) \approx e^{-r\tau} \sum_{k=0}^{N-1} \text{Re} \left[\phi_X \left(\frac{\pi k}{b-a} \right) \cdot \exp \left(-\frac{i\pi a k}{b-a} \right) \right] \cdot H_k, \quad (2.11)$$

with x a function of $S(t_0)$, $\phi_X(u)$ the ChF, and H_K the *payoff coefficients*:

$$H_k := \frac{2}{b-a} \int_a^b V(T, y) \cos \left(\frac{y-a}{b-a} \pi k \right) dy. \quad (2.12)$$

TODO: seguir con los payoffs coefficients

2.5.1. The COS Method Applied To The Heston Model

TODO: rellenar

We need to define the following functions:

$$D_1(u) = \sqrt{(\kappa - \gamma \rho i u)^2 + (u^2 + i u) \gamma^2}, \quad (2.13)$$

$$g(u) = \frac{\kappa - \gamma \rho i u - D_1}{\kappa - \gamma \rho i u + D_1}. \quad (2.14)$$

Using a vector of strikes $\mathbf{K}$, we find the COS formula for the Heston model:

$$V(t_0, \mathbf{x}) \approx \mathbf{K} e^{-r\tau} \cdot \text{Re} \left\{ \sum_{k=0}^{N-1} \varphi_H \left[\frac{\pi k}{b-a}, T; \nu(t_0) \right] U_k \cdot \exp \left[i\pi k \frac{\mathbf{X}(t_0) - a}{b-a} \right] \right\}, \quad (2.15)$$

where no se si es la ChF?

$$\begin{aligned} \varphi_H(t, T; \nu(t_0)) = & \exp \left\{ i u r \tau + \frac{\nu(t_0)}{\gamma^2} \left(\frac{1 - e^{-D_1 \tau}}{1 - g e^{-D_1 \tau}} \right) (\kappa - i \rho \gamma u - D_1) \right\} \cdot \\ & \cdot \exp \left\{ \frac{\kappa \bar{\nu}}{\gamma^2} \left(\tau (\kappa - i \rho \gamma u - D_1) - 2 \log \left(\frac{1 - g e^{-D_1 \tau}}{1 - g} \right) \right) \right\}. \end{aligned} \quad (2.16)$$

2.6. Implied Volatility And Numerical Inversion

2.7. State Of The Art

3. NEURAL NETWORK PRICER FOR HESTON VANILLA OPTIONS

The main objective of this chapter is to construct an ANN-based surrogate pricer for European vanilla options under the Heston stochastic-volatility model. The target quantity is implied volatility, and the surrogate is designed to approximate the forward map

$$(\Theta, m, \tau, r) \mapsto \sigma_{imp}.$$

Reference pricing based on numerical methods (in our pipeline, Heston COS valuation plus implied-volatility inversion) is highly accurate but computationally expensive when repeated many times, as required in calibration and sensitivity workflows. The ANN aims to preserve pricing accuracy while reducing inference time by several orders of magnitude.

Methodologically, this chapter follows the ANN-pricing framework of Liu et al. [1] as the main reference (architecture family, data ranges, and training philosophy), while explicitly documenting the implementation choices adopted in this project.

This chapter focuses exclusively on the pricing network; the calibration architecture is treated separately in Chapter 4.

3.1. Problem Formulation

In this work, vanilla option pricing under Heston is posed as a supervised regression problem. Given a dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad y_i = \sigma_{imp,i},$$

the neural network approximates the map

$$f_{NN} : \mathbb{R}^8 \rightarrow \mathbb{R}, \quad \mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{imp},$$

where $m = S_0/K$ is moneyness and $\tau = T - t$ is time-to-maturity. Each label σ_{imp} is obtained from Heston-consistent synthetic prices through numerical inversion.

The training objective is to estimate parameters $\mathbf{w}$ such that the empirical prediction error over $\mathcal{D}$ is minimized. In this chapter, we use the ANN only as a pricing surrogate, i.e., as a fast approximation of the reference numerical pipeline in the forward map $(\Theta, m, \tau, r) \mapsto \sigma_{imp}$.

We learn implied volatility instead of option price because the IV representation is directly comparable across contracts with different maturities and moneyness levels, and it is the market-standard quotation format for calibration and diagnostics. In contrast, raw

prices mix scale effects (spot/strike/time) that make learning less homogeneous over the domain.

The resulting surrogate is primarily an in-domain approximator. Preliminary out-of-domain checks indicate that the network remains stable for moderate departures from the training bounds, while errors increase in extreme regimes. For this reason, the methodological core of this chapter is developed on the predefined training domain, and the extrapolation behaviour is discussed separately in the generalization and robustness analysis.

3.2. Synthetic Data Formulation

Training requires a large set of labeled input-output pairs. To obtain a controlled and fully reproducible dataset, we use synthetic data generated under the Heston model. For each sampled input vector, the target implied volatility is computed through the numerical pricing pipeline (COS valuation followed by implied-volatility inversion).

The raw variable set includes Heston parameters $\Theta = \{\rho, \kappa, \gamma, \bar{v}, v_0\}$ and contract/market variables $\Sigma = \{K, S_0, \tau, r\}$, which defines a 9-dimensional space. In our setup, we fix $K = 1$ and represent the underlying level through moneyness $m = S_0/K$, leading to an 8-dimensional input vector:

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \in \mathbb{R}^8.$$

A full Cartesian grid is computationally prohibitive in this dimension, since using N points per variable scales as N^9 (already intractable for moderate N). Therefore, we rely on Latin Hypercube Sampling (LHS) to obtain broad coverage of the domain with a fixed number of samples. The specific ranges, constraints, and splitting/preprocessing rules are detailed in the following subsections.

3.2.1. Parameter Ranges And Constraints

The sampling domain combines Heston parameters, contract variables, and market rate. Following the ranges used in our configuration (largely aligned with [1]), the training domain is summarized in the table below.

TABLE 3.1. INPUT RANGES USED FOR SYNTHETIC DATA GENERATION

Parameter	Range
ρ	$[-0.9, 0.0]$
κ	$[0.0, 3.0]$
γ	$[0.01, 0.8]$
$\bar{v}$	$[0.01, 0.5]$
v_0	$[0.05, 0.5]$
m	$[0.6, 1.4]$
τ	$[0.05, 3.0]$
r	$[0.0, 0.05]$

Source: Based on [1] and project configuration choices

To avoid non-physical or numerically unstable samples, we enforce $\tau > 0$ and apply a Feller-type admissibility filter

$$2\kappa\bar{v} - \gamma^2 \geq 0,$$

which helps keep the variance process in a realistic regime for data generation and training.

During synthesis we fix the strike to $K = 1$ and price European put options. Fixing K is equivalent to expressing spot through moneyness, which reduces the dimensionality from nine variables $(\Theta, S_0, K, \tau, r)$ to eight inputs (Θ, m, τ, r) without loss of information for this setup.

3.2.2. Sampling Strategy: Latin Hypercube Sampling

The training of the neural network requires generating a large set of model parameters distributed over a high-dimensional domain. In our case, the parameter space is $\mathbb{R}^8$. A purely random sampling of this space may generate highly populated regions while leaving other regions scarcely explored. As a consequence, the resulting training dataset may be poorly representative, leading to suboptimal training performance.

To avoid this problem, we will use the *Latin Hypercube Sampling* (LHS), a sampling method designed to uniformly cover each dimension. The main idea consists in splitting each dimension of the parameter space into N disjoint subintervals of equal length, and randomly selecting exactly one sample within each subinterval.

Thus, LHS guarantees a one-dimensional marginal distribution along each coordinate, independently of the total dimension of the space. This results in a sampling scheme that is more representative of the parameter space.

This creates a representative distribution sampling of the parameter space. In contrast to a full-grid sampling, the total number of samples N remains fixed, preventing

exponential growth in the number of dimensions.

The use of LHS is supported by the classical design-of-experiments literature, where stratified one-dimensional coverage is shown to improve space-filling behaviour with respect to naive random sampling for a fixed budget [2]. In our implementation, sampling is performed directly in the 8-dimensional input domain with a fixed seed and a target of 10^6 valid samples. This is consistent with standard surrogate-modeling practice, where the quality and coverage of the design points strongly influence final emulator accuracy and robustness [3].

3.2.3. Dataset Splitting And Preprocessing

After generation and filtering, we build a dataset with a target of 10^6 valid samples and split it into train/validation/test subsets with proportions 0.8/0.1/0.1. To preserve the distribution of the target variable across splits, we apply stratification by implied-volatility quantile bins.

In the baseline configuration, no explicit feature or target normalization is applied (Liu-style setup). Therefore, the model is trained directly on the physical scale of the variables $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$ and predicts IV in absolute units.

Data quality control is integrated in the synthesis pipeline. Samples with invalid maturity or failing admissibility checks are removed, and the numerical inversion quality is monitored via the residual $|V_{BS}(\sigma_{imp}) - V_{target}|$, with strict retention thresholds to keep only reliable labels for training.

3.2.4. Numerical Label Generation (COS + Brent)

Each label is produced in two deterministic numerical steps. First, the target option value V_{target} is computed with the Heston COS solver for a given input vector $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$. Second, implied volatility is obtained by solving

$$V_{BS}(\sigma) = V_{target}$$

with a Brent root-finding routine under fixed bounds and tolerance [4].

In our implementation, Brent is configured with strict numerical controls (absolute tolerance 10^{-8} , bounded search interval, and iteration cap), and labels are retained only if the reconstruction error satisfies

$$|V_{BS}(\sigma_{imp}) - V_{target}| \leq 10^{-6}.$$

Hence, the accepted dataset is not affected by uncontrolled observational noise: it is a numerically generated dataset with explicit error bounds on label construction. The remaining training error is therefore dominated by approximation/generalization effects of the ANN rather than by noisy supervision targets. **Not completely sure of this**

3.3. Neural Network Architecture

The pricing surrogate is implemented as a fully connected neural network that approximates a smooth nonlinear map from model/contract inputs to implied volatility. This choice is consistent with the structure of the problem: inputs are low-dimensional continuous features without spatial topology or sequential dependence, and the target is a scalar regression value.

From a function-approximation perspective, a feed-forward multilayer perceptron (MLP) provides enough expressive capacity to capture the interactions between Heston parameters, moneyness, maturity, and rates on the selected domain. Compared with architectures designed for images or sequences (e.g., CNNs or transformers), an MLP is more parsimonious and easier to train for this tabular setting.

3.3.1. Model Class

Following [1], we use a fully connected feed-forward network (FCNN). Let $\mathbf{x} \in \mathbb{R}^8$ be

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r),$$

and let $y = \sigma_{imp} \in \mathbb{R}$ denote the target implied volatility. The model defines a parametric function

$$\hat{\sigma}_{imp} = f_{NN}(\mathbf{x}; \mathbf{w}),$$

with trainable parameters $\mathbf{w}$.

Once $\mathbf{w}$ is fixed after training, inference is deterministic: for the same input $\mathbf{x}$, the network always returns the same prediction $\hat{\sigma}_{imp}$. This property is important for downstream calibration and sensitivity analysis, where numerical consistency across repeated calls is required.

The FCNN acts as a surrogate only at inference time. Target labels are still generated with the numerical pipeline (Heston COS pricing followed by implied-volatility inversion), so the network learns to emulate that reference solver on the sampled domain.

3.3.2. Architecture Details

The architecture used in this chapter is

$$8 \rightarrow 200 \rightarrow 200 \rightarrow 200 \rightarrow 200 \rightarrow 1,$$

with ReLU activation in hidden layers and linear activation in the output layer. In compact form,

$$h^{(1)} = \phi(W^{(1)}x + b^{(1)}), \quad \dots, \quad h^{(4)} = \phi(W^{(4)}h^{(3)} + b^{(4)}), \quad \hat{y} = W^{(5)}h^{(4)} + b^{(5)},$$

where $\phi(\cdot) = \max(0, \cdot)$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled ($p = 0$) and no batch normalization layers are used. This configuration follows a controlled, reproducible design intended to isolate the impact of optimization and data quality choices.

TABLE 3.2. NEURAL NETWORK ARCHITECTURE USED FOR THE ANN PRICER

Hyperparameter	Value
Input dimension	8
Hidden layers	4
Hidden width	200 neurons per layer
Hidden activation	ReLU
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Based on [1]

3.3.3. Output Interpretation

The network output is a scalar implied volatility in annualized units (decimal form), not an option price. Therefore, prediction errors are interpreted directly in IV space, which is the representation used in the calibration workflow and in most volatility-surface diagnostics.

Because the output layer is linear, the model is not hard-constrained to produce strictly positive values for every possible input. In practice, training on a physically admissible domain with positive IV labels strongly reduces this risk inside the sampled region. For downstream use, any out-of-domain prediction can be explicitly checked and, if required, filtered before subsequent numerical steps.

This output definition also preserves differentiability of the surrogate with respect to inputs, which is useful in later chapters when the ANN pricer is embedded in calibration and sensitivity pipelines.

3.4. Training Procedure

Training is fully configuration-driven and follows an epoch-based supervised learning loop. At each epoch, the active optimizer performs parameter updates on the training split, then the model is evaluated on the validation split without gradient computation. For Adam updates, optimization is done with standard mini-batch backpropagation; for

L-BFGS updates, each step is computed through a closure that evaluates the loss and its gradient on the selected batch regime.

Each epoch stores train/validation metrics, current optimizer identity, and learning rate. When enabled, the validation monitor can be either the raw validation loss or a trimmed variant that discards the top-tail hardest samples before aggregation. This provides a robust alternative when a small fraction of outlier points may dominate the monitor value.

Execution device is selected automatically (mps when available, otherwise cpu), and data-shuffling reproducibility is controlled through a fixed generator seed from configuration. During training, two checkpoints are maintained: the *last* checkpoint (always overwritten each epoch) and the *best* checkpoint (updated only when the monitored validation metric improves). The best checkpoint is used as the reference model for post-training evaluation and downstream calibration.

3.4.1. Loss Function

Let $\{(\mathbf{x}_i, \sigma_i)\}_{i=1}^B$ denote a mini-batch, where σ_i is the target implied volatility and $\hat{\sigma}_i = f_{NN}(\mathbf{x}_i; \mathbf{w})$. The baseline objective is mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}}(\mathbf{w}) = \frac{1}{B} \sum_{i=1}^B (\hat{\sigma}_i - \sigma_i)^2.$$

MSE is preferred because it is smooth, differentiable everywhere, and strongly penalizes large deviations, which is desirable when the ANN is later used in calibration loops sensitive to local pricing errors. Although MAE and RMSE are available in the implementation, MSE is used as the main training criterion in the reported baseline.

3.4.2. Optimizer And Hyperparameters

PUEDE IR CAMBIANDO: se pretende mejorar las runs

Several optimizer configurations were tested (Adam, mixed schedules, and `mix_half`). For model selection, we prioritize out-of-sample absolute-error behaviour (MAE and quantile stability), since these metrics better reflect the typical pointwise error level used later in calibration and sensitivity tasks.

Under this criterion, the selected reference ANN is `Liu_like_v01` (Adam), while `MIX_half_v01` is the second-best candidate. Although `MIX_half_v01` achieves a slightly lower validation MSE, `Liu_like_v01` provides lower validation and test MAE, together with better high-quantile absolute errors.

TABLE 3.3. REFERENCE AND RUNNER-UP RUNS USED IN OPTIMIZER SELECTION

Run	Optimizer mode	Val MAE	Test MAE
Liu_like_v01	Adam	4.1132×10^{-4}	4.2925×10^{-4}
MIX_half_v01	Adam $\rightarrow$ L-BFGS	5.1342×10^{-4}	5.2377×10^{-4}

Source: Own elaboration from `outputs/runs/*/metrics/eval_global.csv`

The hyperparameters of the selected reference run (Liu_like_v01) are summarized below.

TABLE 3.4. TRAINING HYPERPARAMETERS OF THE SELECTED RUN (LIU_LIKE_V01)

Hyperparameter	Value
Training mode	Adam
Seed (data-loader generator)	42
Epochs	8000
Train batch size	1024
Validation batch size	All validation samples
Loss	MSE
Adam learning rate	10^{-3}
Adam weight decay	0.0
LR scheduler	StepLR, <code>step_size = 500</code> , $\gamma = 0.5$
Early stopping	Disabled

Source: Own elaboration from run configuration copies

3.4.3. Regularization And Callbacks

PUEDE IR CAMBIANDO: se pretende mejorar las runs

Regularization in the baseline is intentionally light: dropout is disabled ($p = 0$), batch normalization is disabled, and Adam weight decay is set to zero. In addition, feature/target normalization is turned off in the Liu-style reference setup. This design keeps the training protocol close to the reference architecture and isolates the effect of optimizer scheduling and data quality controls.

Callback management includes: (i) checkpointing of both *best* and *last* models, (ii) StepLR scheduling for Adam, and (iii) an early-stopping module available in the pipeline but disabled in the baseline experiments. When enabled, early stopping supports both standard validation loss and trimmed validation loss monitors.

From an experimental standpoint, this callback configuration improves training robustness without over-constraining the optimization path: checkpoints protect against late-epoch degradation, while the Adam-to-L-BFGS transition plus scheduler supports stable convergence. Generalization is then assessed empirically on held-out validation and test splits rather than enforced through strong explicit regularizers.

4. CALIBRATION NEURAL NETWORK FOR THE HESTON MODEL

4.1. Calibration problem formulation

4.2. Input representation: implied volatility surfaces

4.3. Synthetic calibration dataset

4.4. CaNN architecture

5. PINN FOR GREEKS COMPUTATION

6. NEURAL NETWORK PERFORMANCE AND BENCHMARKING

6.1. Pricing Accuracy

6.2. Implied Volatility Accuracy

6.3. Computational Efficiency

6.4. Generalization And Robustness

7. CONCLUSION AND FUTURE WORK

7.1. Market Data

7.2. Exotic Derivatives

7.3. Greeks And Hedging

7.4. Model Extension

7.5. Machine Learning Improvements

BIBLIOGRAPHY

- [1] S. Liu, A. Borovykh, L. A. Grzelak, and C. W. Oosterlee, “A neural network-based framework for financial model calibration,” en, *Journal of Mathematics in Industry*, vol. 9, no. 1, p. 9, Dec. 2019. doi: [10.1186/s13362-019-0066-7](https://doi.org/10.1186/s13362-019-0066-7). Accessed: Oct. 21, 2025. [Online]. Available: <https://mathematicsinindustry.springeropen.com/articles/10.1186/s13362-019-0066-7>.
- [2] M. D. McKay, R. J. Beckman, and W. J. Conover, “A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output From a Computer Code,” en, *Technometrics*, vol. 42, no. 1, pp. 55–61, Feb. 2000. doi: [10.1080/00401706.2000.10485979](https://doi.org/10.1080/00401706.2000.10485979). Accessed: Mar. 21, 2026. [Online]. Available: <http://www.tandfonline.com/doi/abs/10.1080/00401706.2000.10485979>.
- [3] R. B. Gramacy, *Surrogates: Gaussian Process Modeling, Design, and Optimization for the Applied Sciences*. Chapman and Hall/CRC, 2020.
- [4] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007.

APPENDIX A: ANALYSIS OF THE COS METHOD ERROR

no se si dejarlo como apéndice o meterlo en alguna seccion

For the COS method, we have three sources of error:

- The integration range truncation error, ε_1 .
- The series truncation error on $[a, b]$, ε_2 .
- The error related to approximating $\bar{A}_k(x)$ by $\bar{F}_k(x)$, ε_3 (poner ref a alguna ecuacion).

It is important to note that there is no error in the payoff coefficient H_k for plain vanilla European options.

If the integration range $[a, b]$ is chosen sufficiently large, the overall error is dominated by ε_2 (TODO: argumentarlo). Also, ε_2 converges exponentially if $f_X(x) \in C^\infty[a, b]$ (TODO: justificarlo tamb). Otherwise, converges only algebraically.

To choose an optimal integration range we will use the expression (TODO: ref a Oosterlee)

$$[a, b] = \left[(x + \zeta_1) - L \sqrt{\zeta_2 + \sqrt{\zeta_4}}, (x + \zeta_1) + L \sqrt{\zeta_2 + \sqrt{\zeta_4}} \right], \quad (7.1)$$

where $L \in [6, 12]$ is a parameter that depends on the tolerance we want to have, and ζ_i are the cumulants, computed using the expression (2.6).

APPENDIX B: CALIBRATION OF THE HESTON MODEL

TODO: